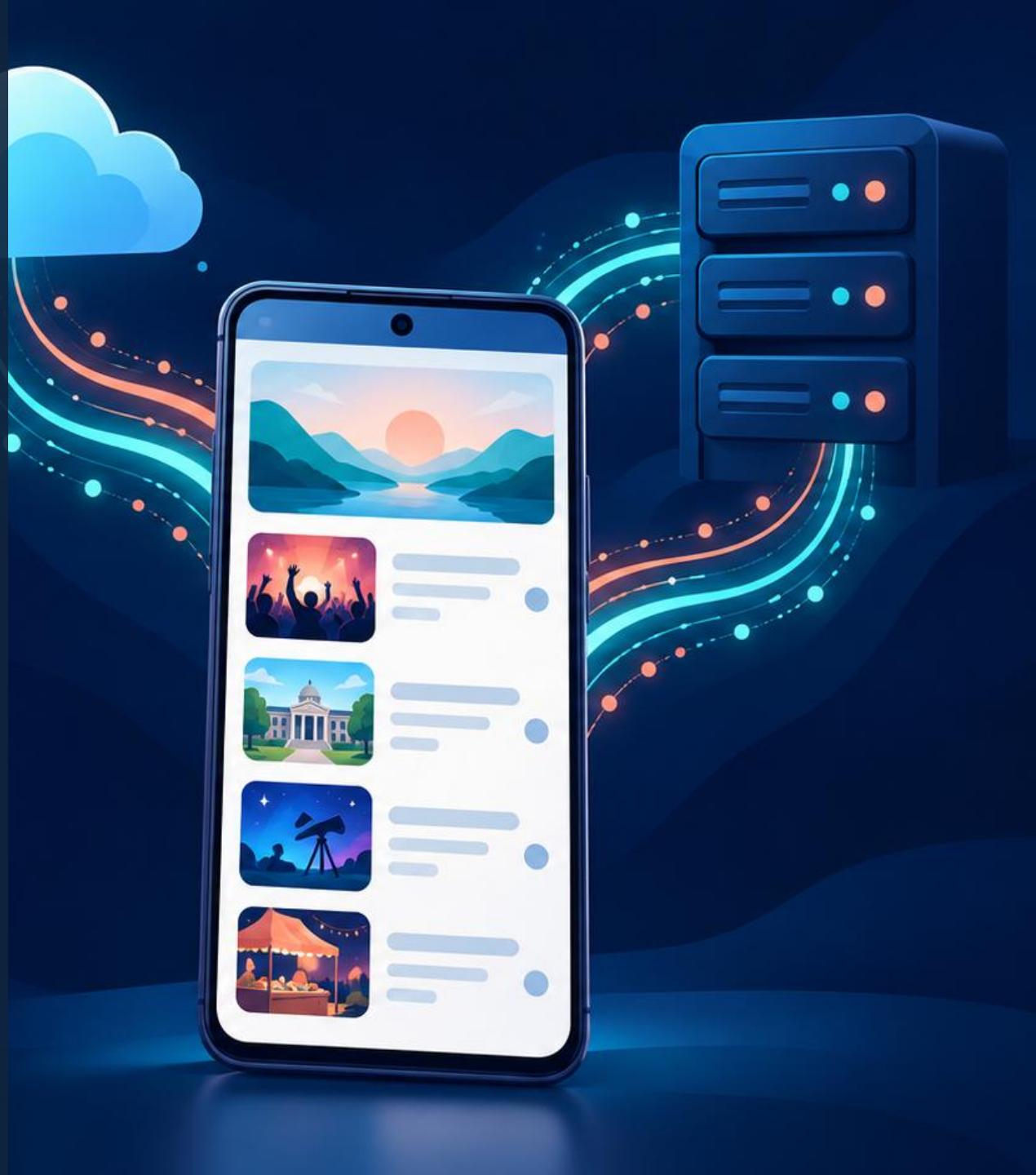
Network data with Retrofit

A remote data source, a repository boundary, and visible loading and error states

Two 75-minute sessions

Android Development



What this module assumes

Networking enters the same architecture already used for local data

Already understood	This module adds	Not yet
Compose UI state ViewModel and repository Room and DataStore viewModelScope	HTTP request/response Retrofit service DTO mapping Remote errors	Auth flows Offline sync Pagination Background refresh

The composable still renders state and reports events. It does not create Retrofit or call an endpoint.

Learning outcomes

One read-only events feed becomes the first remote Campus Companion feature

- Explain an API contract in terms of URL, method, request, response, and status
- Call a simple GET endpoint through Retrofit using a suspend function
- Map a network DTO into UI-ready data through a repository
- Render loading, content, and error states without blocking the screen

The assessment question: “What happens when the server is slow, unavailable, or returns bad data?”

The API contract

A client and a server agree on the shape and meaning of messages

Part	Campus Companion example
Method + URL	GET /events
Response body	A JSON array of campus events
Status code	200 means success; 404 means missing; 500 means server failed
Client model	EventDto is the data shape received over the network

Read the server response before writing Kotlin. The JSON is the external contract.

Connection requirements

The app declares internet access and uses HTTPS endpoints

MANIFEST

```
<!-- AndroidManifest.xml -->
<uses-permission
    android:name="android.permission.INTERNET"
/>
```

Two details

- INTERNET is granted at install time
- Use HTTPS, not cleartext HTTP

For apps targeting Android 9/API 28 and above, cleartext traffic is disabled by default.

Retrofit’s place in the data layer

Retrofit describes HTTP calls. It does not replace your repository.

Retrofit service

Knows endpoint paths and HTTP methods

›

Network data source

Calls the service

›

Repository

Maps data and exposes app operations

›

ViewModel

Turns result into visible UI state

The Retrofit interface belongs below the ViewModel. Compose only receives state and callbacks.

DTOs describe the wire format

A DTO should mirror the JSON you receive, not necessarily the object you show

KOTLIN

```kotlin
@Serializable
data class EventDto(
    val id: String,
    val title: String,
    @SerialName("starts_at") val startsAt: String,
    val location: String? = null
)

data class CampusEvent(
    val id: String,
    val title: String,
    val timeLabel: String,
    val locationLabel: String
)
```

Why map?

- Server names do not leak into UI
- Formatting stays out of the composable
- A changing API has one adaptation point

A Retrofit service uses suspend functions

The network layer provides an operation, not a callback inside the UI

KOTLIN

```kotlin
interface CampusApi {
    @GET("events")
    suspend fun getEvents(): List<EventDto>
}

class EventsRemoteDataSource(
    private val api: CampusApi
) {
    suspend fun getEvents(): List<EventDto> =
        api.getEvents()
}
```

No GlobalScope

The ViewModel starts the work in viewModelScope. Retrofit suspends until the response or failure arrives.

Repository: map and classify results

The repository makes remote data usable by the rest of the app

KOTLIN

```
class EventsRepository(
    private val remote: EventsRemoteDataSource
) {
    suspend fun getEvents(): List<CampusEvent> =
        remote.getEvents().map { dto ->
            CampusEvent(
                id = dto.id,
                title = dto.title,
                timeLabel = dto.startsAt,
                locationLabel = dto.location ?: "TBA"
            )
        }
}
```

Later

The repository can combine the network with Room. Do not add caching or sync to this first networking workshop.

Visible network states

A slow or failed request should produce an intentional UI, not a frozen screen

UI state	What the user sees	Example cause
Loading	Progress indicator or skeleton	Request started
Content	Event list	Valid response with items
Empty	Clear empty-state message	Valid response with no events
Error	Plain message + Retry	No connection, server failure, malformed response

Do not use an empty list to represent every failure. Empty content and an error mean different things.

Failures are not all the same

Classify the problem before choosing a response

Failure	Typical handling
No connection or timeout	Show retryable network error
HTTP 4xx	The request was rejected; show appropriate message
HTTP 5xx	The server failed; invite retry later
Serialization/data mismatch	Log and surface a safe generic error

The first module needs classification and a retry button. Authentication and token refresh belong later.

Workshop: Campus Events feed

Guided build, 45 minutes. Fetch, map, and display a remote list.

Step	Checkpoint
1. Add manifest permission and supplied dependencies	App compiles
2. Create EventDto and CampusApi	Endpoint call has a Kotlin type
3. Create EventsRepository	DTO maps to CampusEvent
4. Call from EventsViewModel	Loading becomes Content or Error
5. Render the state	List, retry, and error message are visible

Success criterion: refresh the feed, then explain each layer touched by that one action.

Workshop failure exercise

Failure is part of the feature, not a bonus task

Try	Students should observe
Disconnect the emulator/network	Error state instead of crash
Temporarily change the base URL	A retryable failure
Return an empty JSON array	A deliberate empty-content message
Restore the valid endpoint	Content returns after Retry

AI prompt pattern: “What category of failure is this? Show the smallest change that preserves our UI-state design.”

Security and project scope

A mobile app is an untrusted client

Do now

- Use HTTPS
- Keep request data minimal
- Treat server data as untrusted

Do later

- Authentication and tokens
- Secure backend authorization
- Secrets and key restrictions

Never place a backend master secret in the Android app. The server must enforce authorization.

Session 2: guided implementation and explanation

75 minutes: one visible remote feature, not a full sync engine

Time	Activity
0–10 min	Architecture and UI-state recap
10–20 min	Instructor implements first service call
20–60 min	Guided Campus Events workshop
60–70 min	Failure exercise and Retry
70–75 min	Pair demo and exit question

Project link and next step

Remote data is one source. The next design question is how it meets local data.

For final projects

- Name one remote feature
- Write its response contract
- Define Loading, Content, and Error

Next architecture step

- Local Room data
- Remote network data
- Repository chooses a source of truth

Exit question: why should EventsScreen receive state rather than a Retrofit service?